

Alumno	Gregorio Ruiz López
Asignatura	Proyecto Intermodular
Curso	2 DAM
Año Académico	2025-2026
Título de la práctica	Trabajo Fin de Grado: Footy IQ



Contenido

1.	Resumen	3
2.	Justificación y objetivos	3
3.	Arquitectura general	3
4.	Tecnologías utilizadas.....	4
5.	Estructura del proyecto	6
6.	Modelo de datos.....	6
7.	Backend	7
8.	Scraper	8
9.	Frontend	9
10.	Seguridad	9
11.	Flujos principales.....	10
12.	API REST	11
13.	Configuración y despliegue.....	13
14.	Acceso desde Android.....	14
15.	Pruebas y validación.....	14
16.	Limitaciones	15
17.	Requisitos funcionales.....	15
18.	Requisitos no funcionales	16
19.	Casos de uso	17
	CU-01 Registro de usuario.....	17
	CU-02 Inicio de sesión.....	17
	CU-03 Importar competición	17
	CU-04 Consultar equipo	18
	CU-05 Guardar favorito	18
20.	Gestión de errores.....	18
21.	Mantenimiento.....	18
22.	Riesgos del proyecto	19
23.	Futuras mejoras.....	20



1. Resumen

Footy IQ es una aplicación web full stack orientada al análisis y consulta de información futbolística. Permite importar competiciones, consultar equipos, jugadores y partidos, guardar favoritos y trabajar con datos persistidos en MongoDB.

El sistema está formado por una interfaz React, una API REST desarrollada con Spring Boot, una base de datos MongoDB y un microservicio scraper en FastAPI que actúa como capa de integración con FotMob. Esta separación permite mantener una arquitectura ordenada y facilita el mantenimiento de cada parte del proyecto.

2. Justificación y objetivos

El proyecto responde a una necesidad habitual en el análisis deportivo: disponer de una herramienta centralizada para consultar información de competiciones, equipos, jugadores y partidos sin depender de búsquedas aisladas en distintas fuentes.

Objetivos principales:

1. Crear una plataforma integrada de análisis futbolístico.
2. Separar responsabilidades entre frontend, backend, base de datos y scraper.
3. Permitir registro e inicio de sesión con autenticación segura.
4. Importar y persistir datos externos.
5. Consultar datos almacenados de forma clara y navegable.
6. Facilitar el seguimiento de entidades mediante favoritos.
7. Permitir el acceso desde PC y desde dispositivos Android en red local.

3. Arquitectura general

La arquitectura se organiza en cuatro servicios principales:

Frontend React: interfaz de usuario y navegación.

- Backend Spring Boot: API central, seguridad, validación y lógica de negocio.
- MongoDB: almacenamiento documental de usuarios y datos futbolísticos.



CES LOPE DE VEGA S.C.A.

CENTRO CONCERTADO BACHILLERATO Y CICLOS FORMATIVOS

- Scraper FastAPI: consulta y transformación de datos externos.

Flujo general:

1. El usuario accede al frontend.
2. El frontend autentica contra el backend.
3. El backend valida el JWT en rutas protegidas.
4. Para datos persistidos, el backend consulta MongoDB.
5. Para datos externos, el backend consulta el scraper.
6. El scraper obtiene información de FotMob.
7. El frontend renderiza paneles, fichas, tablas y gráficos.

4. Tecnologías utilizadas

Backend

- Java 21.
- Spring Boot 3.4.3.
- Spring Web.
- Spring Security.
- Spring Data MongoDB.
- JWT.
- Springdoc OpenAPI.
- Maven.

Frontend

- React 18.
- TypeScript.
- Vite 5.



- Axios.
- React Router DOM.
- Recharts.
- Lucide React.
- jsPDF.

Scraper

- Python 3.
- FastAPI.
- Uvicorn.
- HTTPX.
- BeautifulSoup4.
- Pydantic.
- APScheduler.
- Playwright mediante CDP para reutilizar una sesión real de Chrome cuando FotMob lo requiere.

Infraestructura

- Docker.
- Docker Compose.
- MongoDB 7.
- PowerShell para scripts de desarrollo.



5. Estructura del proyecto

```
backend/  
  src/main/java/com/footyiq/api/  
    auth/  
    config/  
    controller/  
    dto/  
    model/  
    repository/  
    service/  
frontend/  
  src/  
    components/  
    context/  
    pages/  
    services/  
    types/  
    utils/  
scraper/  
  app/  
    main.py  
    fotmob_client.py  
    browser_client.py  
    league_catalog.py  
    scheduler.py  
scripts/  
  dev-up.ps1  
docker-compose.yml  
run-local.ps1  
abrir-footyiq.bat  
reiniciar-backend.bat
```

6. Modelo de datos

Colecciones principales de MongoDB:

- users: cuentas de usuario con nombre, email, hash de contraseña y rol.
- dashboard_snapshots: resúmenes del dashboard.
- competitions: competiciones importadas desde FotMob.
- teams: equipos importados.
- players: jugadores importados.



CES LOPE DE VEGA S.C.A.
CENTRO CONCERTADO BACHILLERATO Y CICLOS FORMATIVOS

- matches: partidos importados.
- imports: historial de importaciones.
- favorites: elementos guardados por usuario.
- custom_teams: equipos creados por el usuario.

Relaciones principales:

- Un usuario puede tener favoritos y equipos personalizados.
- Una competición agrupa partidos y equipos mediante identificadores externos.
- Un equipo puede pertenecer a varias competiciones.
- Un jugador importado pertenece a un equipo externo.
- Un favorito se identifica por usuario, tipo y externalId.

7. Backend

El backend es el núcleo de la aplicación. Expone la API REST bajo /api, gestiona autenticación JWT, aplica seguridad stateless y coordina el acceso a MongoDB y al scraper.

Paquetes destacados:

- auth: filtro JWT y servicio de tokens.
- config: CORS, seguridad, propiedades y configuración MVC.
- controller: endpoints REST.
- dto: objetos de transferencia.
- model: entidades documentales.
- repository: repositorios MongoDB.
- service: lógica de negocio.

Servicios principales:

- AuthService: registro, login, hash de contraseña y generación de JWT.



CES LOPE DE VEGA S.C.A.

CENTRO CONCERTADO BACHILLERATO Y CICLOS FORMATIVOS

- DashboardService: consulta y actualización de resúmenes.
- FotmobDataService: comunicación con el scraper.
- DataIngestionService: importación de ligas a MongoDB.
- StoredDataQueryService: consulta de datos persistidos.
- TeamService: gestión de equipos propios.
- FavoriteService o lógica equivalente de favoritos.

8. Scraper

El scraper es un microservicio FastAPI que concentra el acceso a FotMob. Su objetivo es aislar la obtención de datos externos para que el backend no dependa directamente de la estructura del proveedor.

Endpoints principales:

- GET /health
- GET /browser/health
- GET /fotmob/dashboard
- GET /fotmob/catalog/leagues
- GET /fotmob/league/{league_id}
- GET /fotmob/team/{team_id}
- GET /fotmob/team/{team_id}/players
- GET /fotmob/player/{player_id}
- GET /fotmob/match/{match_id}
- GET /fotmob/league/{league_id}/matches
- GET /fotmob/matches?date=yyyymmdd



CES LOPE DE VEGA S.C.A.

CENTRO CONCERTADO BACHILLERATO Y CICLOS FORMATIVOS

El scraper incluye un mecanismo de integración con Chrome real mediante CDP. Esto permite resolver manualmente la protección de Cloudflare/FotMob cuando aparece y reutilizar la sesión desde el servicio.

9. Frontend

El frontend es una SPA desarrollada con React y TypeScript. Si no existe token, muestra la pantalla de login; si el usuario está autenticado, renderiza la aplicación principal con barra superior y rutas protegidas.

Rutas principales:

- /: Dashboard.
- /match: análisis de partido.
- /equipos: consulta de equipos.
- /teams: gestión de equipos propios.
- /player: ficha de jugador.
- /competition: competición.
- /data-hub: importación y datos.
- /favorites: favoritos.

La capa frontend/src/services/api.ts centraliza llamadas HTTP con Axios y gestiona el token JWT en localStorage. Para facilitar acceso desde Android en red local se usa una URL relativa (/api) y un proxy de Vite hacia el backend local.

10. Seguridad

La seguridad se basa en:

- Registro e inicio de sesión.
- Contraseñas hasheadas mediante BCrypt.
- Emisión de JWT.
- Almacenamiento del token en localStorage.



- Envío del token en cabecera Authorization: Bearer.
- Filtro JWT en backend.
- Sesiones stateless.
- Separación de rutas públicas y privadas.

Aspectos que deberían reforzarse en producción:

- Usar HTTPS.
- Sustituir el JWT_SECRET de desarrollo por un secreto seguro.
- Configurar CORS según dominio real.
- Definir política de expiración y renovación de tokens.
- Revisar validaciones de archivos subidos.

11. Flujos principales

Registro e inicio de sesión

El usuario introduce credenciales.

El frontend envía la petición al backend.

El backend valida datos.

Se genera un JWT.

El frontend guarda el token y permite acceder a la aplicación.

Importación de competición

El usuario entra en Data Hub.

Selecciona una liga.

El frontend llama al backend.

El backend crea un registro de importación.

El backend solicita datos al scraper.



Se guardan competiciones, partidos, equipos y jugadores.

El historial queda marcado como SUCCESS o FAILED.

Consulta de datos

El usuario abre una sección como Competition, Equipos, Player o Match.

El frontend solicita datos al backend.

El backend consulta MongoDB o el scraper según proceda.

La información se devuelve al frontend para su representación.

Favoritos

El usuario guarda un jugador, equipo o partido.

El backend evita duplicados por usuario, tipo e identificador externo.

La sección Favoritos muestra los elementos guardados.

12. API REST

Autenticación

- POST /api/auth/register
- POST /api/auth/login

Salud

- GET /api/public/health

Dashboard

- GET /api/dashboard/summary
- POST /api/dashboard/refresh

FotMob proxy

- GET /api/fotmob/catalog/leagues
- GET /api/fotmob/league/{leagueId}
- GET /api/fotmob/team/{teamId}



- GET /api/fotmob/team/{teamId}/players
- GET /api/fotmob/player/{playerId}
- GET /api/fotmob/match/{matchId}
- GET /api/fotmob/matches?date=yyyymmdd
- GET /api/fotmob/league/{leagueId}/matches

Data Hub

- POST /api/data/import/league/{leagueId}
- DELETE /api/data/competition/{leagueId}
- GET /api/data/imports
- GET /api/data/competitions
- GET /api/data/competition/{leagueId}/matches
- GET /api/data/competition/{leagueId}/teams
- GET /api/data/team/{teamId}/players
- GET /api/data/player/{playerId}
- GET /api/data/match/{matchId}

Equipos propios

- GET /api/teams
- POST /api/teams
- POST /api/teams/{teamId}/players

Favoritos

- GET /api/favorites
- GET /api/favorites?type=PLAYER|TEAM|MATCH
- POST /api/favorites
- DELETE /api/favorites/{type}/{externalId}



Herramientas scraper

- GET /api/scraper/browser/status
- POST /api/scraper/browser/launch

13. Configuración y despliegue

Variables principales:

- SPRING_DATA_MONGODB_URI: URI de MongoDB.
- SCRAPER_BASE_URL: URL del scraper.
- JWT_SECRET: clave de firma JWT.
- UPLOADS_PATH: carpeta de archivos subidos.
- VITE_API_URL: URL base de la API para el frontend.
- SCRAPER_PORT: puerto del scraper.

Despliegue local con Docker:

```
docker compose up --build
```

Desarrollo local:

```
powershell -ExecutionPolicy Bypass -File .\run-local.ps1
```

Arranque completo:

```
.\abrir-footyiq.bat
```

Reinicio rápido de backend:

```
.\reiniciar-backend.bat
```

Scraper en modo local:

```
cd scraper
```

```
python -m uvicorn app.main:app --port 8001
```



CES LOPE DE VEGA S.C.A.

CENTRO CONCERTADO BACHILLERATO Y CICLOS FORMATIVOS

Se recomienda no usar --reload en Windows cuando se emplea Playwright/CDP, ya que puede provocar errores internos en algunas combinaciones de Python y bucle de eventos.

14. Acceso desde Android

La aplicación puede abrirse desde Android si el PC y el móvil están en la misma red.

Configuración aplicada:

- Vite escucha en 0.0.0.0.
- El frontend usa VITE_API_URL=/api.
- Vite redirige /api hacia http://localhost:8080 en el PC.

Uso:

1. Levantar servicios en el PC.
2. Obtener IP local con ipconfig.
3. Abrir http://<IP_DEL_PC>:5173 en Android.
4. Permitir tráfico entrante en firewall si fuera necesario.

15. Pruebas y validación

Pruebas recomendadas:

- Backend: mvn test.
- Frontend: npm run build.
- Scraper: comprobar /health y endpoints principales.
- Integración: validar login, importación de liga, consulta de equipos, jugadores y partidos.
- Seguridad: comprobar que rutas privadas rechazan peticiones sin token.
- Acceso LAN: probar desde Android mediante IP local.



16. Limitaciones

- La disponibilidad de datos depende de FotMob.
- Cambios en el proveedor externo pueden afectar al scraper.
- La cuenta demo solo rellena credenciales en pantalla; si el usuario no existe, debe registrarse.
- El secreto JWT incluido es de desarrollo.
- Algunas vistas dependen de identificadores externos correctos.
- La ejecución local requiere que los servicios estén levantados en los puertos esperados.

17. Requisitos funcionales

Autenticación

- Registrar usuario.
- Iniciar sesión.
- Generar token JWT.
- Cerrar sesión eliminando el token local.

Dashboard

- Mostrar resumen inicial.
- Consultar snapshot almacenado.
- Refrescar datos desde el scraper.
- Mantener fallback si el scraper no responde.

Data Hub

- Mostrar catálogo de ligas.
- Importar liga por identificador externo.
- Guardar historial de importación.



- Consultar competiciones almacenadas.
- Eliminar competiciones importadas.

Consulta futbolística

- Consultar ligas, equipos, jugadores y partidos.
- Navegar entre entidades relacionadas.
- Presentar la información de forma comprensible.

Favoritos

- Guardar jugadores, equipos y partidos.
- Evitar duplicados.
- Listar favoritos por usuario.
- Eliminar favoritos.

Equipos propios

- Crear equipos personalizados.
- Adjuntar escudo opcional.
- Añadir jugadores.

18. Requisitos no funcionales

Seguridad

- Las rutas privadas deben exigir JWT.
- Las contraseñas deben guardarse hasheadas.
- El backend debe ser stateless.
- El secreto JWT debe configurarse mediante variable de entorno en producción.

Usabilidad

- La navegación debe ser clara.



- Las acciones principales deben estar visibles.
- Los mensajes de error deben ser comprensibles.
- La aplicación debe poder usarse tanto desde PC como desde Android en red local.

Mantenibilidad

- Separación entre frontend, backend, scraper y base de datos.
- Servicios backend organizados por responsabilidad.
- Scraper aislado para reducir impacto de cambios externos.
- Configuración mediante variables de entorno.

19. Casos de uso

CU-01 Registro de usuario

Actor: usuario no autenticado.

Precondición: backend y MongoDB activos.

Resultado: usuario creado y token devuelto.

CU-02 Inicio de sesión

Actor: usuario registrado.

Precondición: credenciales válidas.

Resultado: acceso a la aplicación.

CU-03 Importar competición

Actor: usuario autenticado.

Precondición: scraper disponible.

Resultado: competición, partidos, equipos y jugadores almacenados.



CU-04 Consultar equipo

Actor: usuario autenticado.

Precondición: competición importada o identificador disponible.

Resultado: visualización de plantilla, partidos y datos del equipo.

CU-05 Guardar favorito

Actor: usuario autenticado.

Precondición: entidad consultada.

Resultado: favorito guardado sin duplicados.

20. Gestión de errores

El frontend muestra mensajes sencillos cuando una operación falla. El backend devuelve códigos HTTP adecuados para credenciales inválidas, recursos inexistentes o errores de validación. El scraper puede fallar por conexión, bloqueo temporal, cambio externo o respuesta inesperada.

Errores habituales:

- 401 Unauthorized: token ausente o incorrecto.
- 400 Bad Request: datos obligatorios ausentes.
- 404 Not Found: recurso inexistente.
- 409 Conflict: usuario o email duplicado.
- 500 Internal Server Error: fallo no controlado.

21. Mantenimiento

Para mantener el proyecto:

- Ejecutar npm run build tras cambios en frontend.
- Ejecutar mvn test tras cambios en backend.
- Probar /health del scraper.



- Revisar logs cuando falle una importación.
- Mantener documentación actualizada tras cambios funcionales.

Si FotMob cambia su estructura:

1. Probar endpoint del scraper directamente.
2. Revisar logs.
3. Ajustar fotmob_client.py.
4. Probar backend.
5. Validar frontend.

22. Riesgos del proyecto

Dependencia externa

El mayor riesgo es la dependencia de FotMob. Un cambio en su estructura o protección puede afectar al scraper.

Mitigación:

- Mantener el scraper separado.
- Registrar errores.
- Conservar datos ya importados.
- Añadir caché o reintentos en futuras versiones.

Seguridad de entorno local

El proyecto incluye valores de desarrollo.

Mitigación:

- Variables de entorno en producción.
- HTTPS.
- Rotación de secretos.



- CORS restringido.

Volumen de datos

Importar muchas ligas puede aumentar el tamaño de MongoDB.

Mitigación:

- Índices.
- Limpieza de ligas no utilizadas.
- Paginación en listados grandes.

23. Futuras mejoras

- Despliegue cloud con HTTPS.
- Roles avanzados de usuario.
- Panel de administración.
- Tests E2E.
- Caché persistente de fichas externas.
- Comparativas entre jugadores y equipos.
- Exportaciones PDF más completas.
- Observabilidad con logs, métricas y trazas.

La arquitectura actual permite evolucionar el proyecto sin rehacer sus componentes principales.